

Documentação do MVP

Navegação inteligente com roteamento multi-critério

Projeto: TrafficMind

Versão: 1.0.0-mvp

Data: Julho de 2026

Foco geográfico: São Paulo, Brasil

1. Sumário Executivo

O TrafficMind é um aplicativo web mobile-first de navegação inteligente que diferencia-se dos concorrentes tradicionais (Waze, Google Maps) por não se limitar ao cálculo da rota mais rápida. Em vez disso, o sistema calcula múltiplas rotas alternativas simultaneamente e as classifica através de um score composto e transparente que combina quatro dimensões: tempo estimado de chegada (ETA), nível de trânsito simulado, número de cruzamentos e complexidade da via.

Este documento descreve o estado atual do Minimum Viable Product (MVP) desenvolvido em julho de 2026, incluindo a arquitetura técnica completa, as funcionalidades implementadas, as dificuldades encontradas durante o desenvolvimento, a stack tecnológica adotada, os custos de operação e uma análise de viabilidade comercial. O objetivo é fornecer uma visão clara e honesta do que foi construído, do que falta construir e do valor de mercado do produto em seu estágio atual.

O MVP está funcional e acessível via deploy público. Ele demonstra end-to-end o fluxo de busca de destino, geolocalização do usuário, cálculo de cinco rotas alternativas com estratégias distintas, exibição em mapa interativo com tema escuro e ranking por score composto. A arquitetura foi desenhada seguindo os princípios de Clean Architecture, com separação clara entre camadas de domínio, aplicação, infraestrutura e apresentação, facilitando a substituição de componentes individuais (como o motor de trânsito simulado por um motor de IA real) sem impacto no restante do sistema.

URL de Deploy (Preview)

A aplicação está em execução contínua no ambiente de preview. Para testes com localização pré-definida (bypassando o prompt de GPS), adicione o parâmetro `?origin=lat,lng` à URL, por exemplo: `?origin=-23.5613,-46.6565` (Av. Paulista, São Paulo).

2. Funcionalidades Implementadas no MVP

2.1 Mapa Interativo

Renderização de mapa vetorial usando MapLibre GL JS sobre tiles raster escuros da CARTO baseados em OpenStreetMap. O mapa suporta pan, zoom, gestos touch e mouse, escala métrica discreta e controles de zoom flutuantes. Sobre o mapa são desenhados: marcador de localização do usuário (com pulso animado em ciano), marcador de destino (pin âmbar clássico), até cinco polylines de rotas alternativas com cores distintas por estratégia e marcadores circulares coloridos para cada leitura de trânsito (verde abaixo de 30, âmbar abaixo de 55, laranja abaixo de 75, vermelho acima disso).

2.2 Geolocalização (GPS)

Integração com a Geolocation API do navegador via hook React customizado (useGeolocation). O app solicita permissão automaticamente quando o usuário define um destino, exibe indicador de progresso (Passo 1, 2 e confirmação) e oferece fallback de "Usar o centro do mapa como partida" para usuários que negam permissão ou estão em dispositivos sem GPS. O watchPosition mantém a localização atualizada continuamente com alta precisão.

2.3 Busca de Destino

Busca de endereços via API Nominatim do OpenStreetMap com debounce de 350ms para evitar requisições excessivas durante a digitação. Os resultados são bias-adjusted para Brasil (countryCodes igual a br) e exibidos em dropdown com label primário e endereço secundário. Suporta também long-press no mapa para drop de pin com reverse geocoding automático (transforma coordenada em endereço legível).

2.4 Cálculo de Rotas Multi-Estratégia

O backend calcula cinco rotas alternativas entre origem e destino em uma única chamada ao OSRM com alternatives igual a true. Cada rota é etiquetada com uma estratégia distinta e enriquecida com metadados de trânsito antes de ser devolvida ao frontend. As cinco estratégias implementadas são:

Estratégia	Label (PT)	Descrição	Cor no Mapa
fastest	Mais rápida	Minimiza tempo total de viagem	#06B6D4 (ciano)
shortest	Mais curta	Minimiza distância total	#10B981 (verde)
scenic	Cênica	Placeholder, prefere vias mais calmas	#F59E0B (âmbar)
least_turns	Menos curvas	Minimiza manobras e cruzamentos	#A855F7 (roxo)
experimental	IA experimental	Blend com peso em trânsito (slot de IA)	#EF4444 (vermelho)

2.5 Sistema de Score Composto

Em vez de ordenar rotas apenas por ETA, o TrafficMind calcula um RouteScore que combina quatro dimensões com pesos configuráveis. A fórmula é:

$$\text{score} = \text{ETA} \times \text{etaWeight} + \text{trafficLevel} \times \text{trafficWeight}$$

```

+ intersectionCount × intersectionPenalty
+ accidents × accidentPenalty (zero no MVP)
+ roadComplexity × roadComplexityWeight

```

Menor score vence. Os pesos padrão são: etaWeight igual a 1 (1 segundo igual a 1 ponto), trafficWeight igual a 6, intersectionPenalty igual a 25, roadComplexityWeight igual a 4. Cada estratégia pode sobrescrever pesos (ex: least_turns usa intersectionPenalty igual a 60). O breakdown do score é exibido ao usuário em barras proporcionais, tornando o motor de decisão transparente e auditável.

2.6 Motor de Trânsito Simulado

O TrafficEngine gera leituras de trânsito deterministicamente para um catálogo de doze vias reais de São Paulo (Av. Paulista, Marginal Pinheiros, Marginal Tietê, Av. Faria Lima, Av. Rebouças, Av. 23 de Maio, Minhocão, R. Augusta, etc.). Para cada via, o motor aplica uma curva bimodal de pico matinal (08h) e vespertino (17h) com amplitude baseada no perfil da via (Marginal Pinheiros pico igual a 90, ruas residenciais dos Jardins pico igual a 30). Adiciona ruído sinusoidal determinístico para parecer vivo sem ser caótico. As leituras são cacheadas por 30 segundos.

2.7 Slot de IA (TrafficPredictor)

Interface TrafficPredictor define o contrato que qualquer modelo de IA futuro deve implementar: predictRoadWeight, predictCongestion e predictETA. O MVP inclui uma implementação mock (MockTrafficPredictor) que devolve valores plausíveis derivados do TrafficEngine. Substituir o mock por um modelo real de Machine Learning (ex: Graph Neural Network treinada em dados históricos) requer mudança em uma única linha no container de injeção de dependência.

2.8 Estatísticas e Painel de Score

Painel inferior colapsável (bottom sheet) em três estados: recolhido (apenas rota recomendada), meio (lista de alternativas) e expandido (estatísticas completas e breakdown). Mostra seis métricas por rota: distância, ETA, nível de trânsito, número de cruzamentos, estimativa de combustível e complexidade da via. O breakdown do score é visualizado em barras horizontais coloridas por componente.

2.9 Interface em Português com Fluxo Guiado

UI 100% em português brasileiro com fluxo de três passos guiado por indicador visual numérico (1, 2 e confirmação). Passo 1: "Para onde vamos?" com dica de busca. Passo 2: "De onde você sai?" com dois botões grandes ("Usar minha localização atual" e "Usar o centro do mapa como partida"). Passo 3: rotas calculadas com badge de Recomendada. Erros de GPS são traduzidos para mensagens amigáveis ("Permissão negada" em vez de "User denied Geolocation").

3. Arquitetura Técnica

3.1 Clean Architecture

O código segue Clean Architecture com quatro camadas estritamente separadas. A regra de dependência é unidirecional: as camadas externas dependem das internas, nunca o contrário. Toda lógica de negócio vive no domínio; controllers e routes são finos e não contêm regras de negócio.

Camada	Responsabilidade	Localização
Domain	Entidades, value objects, interfaces (ports). Zero dependências externas.	src/server/domain/
Application	Casos de uso e serviços de domínio (RouteScoringService, TrafficEngine, use cases).	src/server/application/
Infrastructure	Adaptadores concretos: OsmRoutingEngine, NominatimGeocodingService, MockTrafficPredictor.	src/server/infrastructure/
Presentation	API routes (Next.js) e componentes React. Sem lógica de negócio.	src/app/api/ e src/components/

3.2 Diagrama de Fluxo de Dados

```

Usuário (browser)
| digita destino
v
SearchPanel -> useDebounceGeocode -> /api/geocode
| escolhe resultado
v
NavigationStore (setDestination)
| destino setado, sem origem
v
useGeolocation.request() -> prompt de permissão
| origem obtida
v
useCalculateRoutes -> /api/route
| POST
v

```

```

calculateRoutesUseCase -> OsmRoutingEngine
  | OSRM API (alternatives=true)
  v
enrich(raw, strategy) -> TrafficRepository.getTrafficForRegion
  | trânsito simulado
  v
RouteScoringService.score(route) -> Route[]
  | rank por score
  v
Response -> NavigationStore.setRoutes
  | re-render
  v
NavigationMap (polylines) + RouteSheet (estatísticas)

```

4. Stack Tecnológica

4.1 Frontend

Tecnologia	Versão	Uso
Next.js	16.1	Framework React com App Router, API routes serverless
React	19.0	Biblioteca de UI
TypeScript	5.x	Tipagem estática em todo o código
Tailwind CSS	4.x	Estilização utility-first com tema dark customizado
shadcn/ui	New York	Componentes acessíveis (Radix UI primitives)
MapLibre GL JS	5.24	Renderização de mapa vetorial open-source
TanStack Query	5.82	Cache e sincronização de estado server-side
Zustand	5.0	Estado de sessão do cliente (origem, destino, rotas)
Framer Motion	12.x	Animações sutis de entrada e transição

Tecnologia	Versão	Uso
Lucide React	0.525	Ícones SVG consistentes
Sonner	2.0	Toast notifications com dedup

4.2 Backend (API Routes Next.js + NestJS de referência)

Tecnologia	Versão	Uso
Next.js API Routes	16	Endpoints REST serverless (/api/route, /api/geocode, etc.)
Zod	4.0	Validação de schema de request e response
Node.js	20	Runtime JavaScript (LTS)
NestJS (referência)	10.4	Implementação alternativa para deploy Docker
Swagger/OpenAPI	3.0	Documentação de API em /public/docs/openapi.json

4.3 Infraestrutura e Serviços Externos

Componente	Tecnologia	Status no MVP
Motor de roteamento	OSRM (router.project-osrm.org)	Serviço público, substituível por GraphHopper self-hosted
Geocoding	Nominatim (nominatim.openstreetmap.org)	Serviço público, substituível por Mapbox ou Google
Tiles de mapa	CARTO dark basemaps	Serviço gratuito com rate limit
Banco de dados	PostgreSQL 16 + PostGIS 3.4	Configurado no docker-compose, não usado em runtime no MVP
Cache	Redis 7	Configurado no docker-compose, não usado em runtime no MVP
Containerização	Docker + Docker Compose	docker-compose.yml completo com 6 serviços

4.4 Endpoints de API

Método	Rota	Descrição
POST	/api/route	Calcula e ranqueia 5 rotas alternativas entre origem e destino
GET	/api/geocode?q=	Busca endereços via Nominatim (debounce 350ms no client)
GET	/api/reverse-geocode?lat=&lng=	Reverse geocoding de coordenada para label
GET	/api/traffic? south=&west=&north=&east=	Leituras de trânsito simulado para a região visível
GET	/api/health	Probe de liveness e readiness para docker
GET	/docs/openapi.json	Spec OpenAPI 3.0 completa

5. Dificuldades Encontradas Durante o Desenvolvimento

Esta seção documenta os problemas reais enfrentados durante o desenvolvimento do MVP, com transparência sobre causa raiz e correção aplicada. Estes pontos são importantes para o cliente entender a complexidade real do produto e para o time técnico evitar regressões futuras.

5.1 Bug do containerRef Não Atribuído

Sintoma: usuário clicava em um endereço no dropdown de busca e o destino não era setado — o dropdown fechava mas nada acontecia. Causa raiz: o containerRef foi declarado no componente SearchPanel mas nunca atribuído a nenhuma div no JSX. O handler de outside-click verificava containerRef.current contém e.target, que sempre retornava undefined (falsy) porque o ref era null, fazendo o handler tratar qualquer clique como "fora" e fechar o dropdown antes do onClick do botão disparar. Correção: adicionar ref igual a containerRef na div externa. Esse bug passou pela primeira rodada de QA porque .click() programático (usado nos testes) não dispara mousedown, então o handler não atrapalhava.

5.2 Bug do Mapa "Do Mundo Inteiro"

Sintoma: ao calcular uma rota longa (ex: Av. Paulista para Aeroporto de Guarulhos, aproximadamente 30km), o mapa afastava o zoom para mostrar origem e destino simultaneamente, dando a impressão

de "mapa do mundo inteiro". Causa raiz: a chamada `map.fitBounds` com padding simétrico de 100px e `maxZoom` 15 em rotas longas resultava em zoom muito baixo. Correção: padding assimétrico (top 180, bottom 280, left 60, right 60) com `maxZoom` 14, garantindo que o painel inferior não sobreponha a rota e o zoom não afaste demais.

5.3 Rate Limiting do OSRM Público

Sintoma: a primeira requisição `POST /api/route` às vezes retornava zero rotas, e as subsequentes funcionavam. Causa raiz: o servidor público OSRM impõe rate limit por IP e ocasionalmente responde com code Ok mas routes vazio quando sobrecarregado. Correção: adicionado retry interno com backoff de 200ms na primeira falha, garantindo que falhas transientes não cheguem ao usuário.

5.4 Estados Vazios Não-Contextuais

Sintoma: usuário selecionava destino mas a tela mostrava "No routes yet" sem explicar que faltava definir a origem (GPS). Causa raiz: o RouteSheet tinha apenas um estado vazio genérico. Correção: três estados vazios contextuais — sem destino ("Para onde vamos?"), com destino mas sem origem ("De onde você sai?" com CTAs), e calculando ("Calculando suas rotas..."). Adicionado também `auto-request` de GPS quando o destino é setado.

5.5 Toasts Duplicados de Erro de GPS

Sintoma: quando o usuário negava permissão de GPS, apareciam 8 toasts idênticos de erro empilhados. Causa raiz: o `useEffect` que escutava `geo.error` tinha `geo` (objeto inteiro) como dependência, e como o objeto mudava de identidade a cada render, o effect disparava repetidamente. Correção: usar id estável no `toast.error` (id `geo-error`) para que o Sonner substitua em vez de empilhar, e depender apenas de `geo.error` (string primitiva).

5.6 Jargão Técnico Acessível a Leigos

Sintoma: usuário não-técnico relatou dificuldade de uso — a UI estava em inglês com termos como "long-press", "crosshair", "GPS", "map center". Correção: tradução completa para português brasileiro, remoção de jargão ("long-press" virou "arraste o mapa", "crosshair" virou "Minha localização"), botões grandes com texto (não só ícone), fluxo guiado de 3 passos com indicador visual, dicas contextuais ("Dica: arraste o mapa para posicionar...").

6. Custos de Operação

6.1 Custos Atuais (MVP em preview)

O MVP roda atualmente em ambiente de preview sem custo direto de infraestrutura. Os serviços externos consumidos são todos gratuitos com rate limits generosos para tráfego baixo:

Serviço	Custo Mensal	Limite
OSRM público (router.project-osrm.org)	R\$ 0	Aproximadamente 1 req/s por IP
Nominatim público	R\$ 0	1 req/s (policy estrita)
CARTO dark tiles	R\$ 0	Rate limit não documentado
Hospedagem preview (Vercel-like)	R\$ 0	Incluído no ambiente
Total MVP	R\$ 0 por mês	Suficiente para demo e POC

6.2 Custos Estimados para Produção

Para um deploy de produção com usuários reais, os custos mudam significativamente. As estimativas abaixo consideram tráfego moderado (aproximadamente 1.000 usuários ativos por dia, 10.000 requisições de rota por dia):

Item	Custo Mensal Estimado	Observação
VPS (backend + frontend, 4 vCPU, 8GB RAM)	R\$ 200 a 400	DigitalOcean, Hetzner ou AWS Lightsail
OSRM self-hosted (com OSM Brasil)	R\$ 0 (incluído no VPS)	Requer aproximadamente 10GB RAM para São Paulo
PostgreSQL + PostGIS	R\$ 0 (incluído no VPS)	Volume persistente aproximadamente 20GB
Redis	R\$ 0 (incluído no VPS)	Cache de rotas e trânsito
Trânsito real (TomTom ou Here API)	R\$ 2.000 a 8.000	Opcional, sem isso continua simulação
Tiles de mapa (Mapbox ou CARTO paid)	R\$ 100 a 500	Opcional, free tier pode bastar

Item	Custo Mensal Estimado	Observação
Domínio + SSL	R\$ 15	Certbot Let's Encrypt grátis
Total produção (sem trânsito real)	R\$ 215 a 415 por mês	Custo mínimo viável
Total produção (com trânsito real)	R\$ 2.215 a 8.415 por mês	Custo com dados profissionais

Atenção sobre Custo de Trânsito Real

O maior custo recorrente de um app de navegação é o feed de trânsito em tempo real. Sem isso, o app usa simulação (como o MVP atual). Com trânsito real, o custo mensal salta de centenas para milhares de reais. Esta decisão deve ser alinhada com o cliente antes de fechar escopo.

7. Análise de Valor de Mercado

Esta seção estima o valor de mercado de um sistema no estágio atual do TrafficMind, considerando desenvolvimento por um engenheiro sênior assistido por IA (GitHub Copilot, Claude, etc.). Os valores são referências para o mercado brasileiro de desenvolvimento de software sob encomenda em 2026.

7.1 Cálculo por Horas de Desenvolvimento

O MVP atual representa aproximadamente 45 a 55 horas de trabalho efetivo de um engenheiro sênior com assistência de IA. Considerando o rate hora de um sênior no Brasil (R\$ 150 a 250 por hora CLT equivalente, ou R\$ 300 a 500 por hora como freelancer ou consultor), o custo de reprodução do MVP seria:

Item	Horas	Rate Sênior+IA (R\$/h)	Subtotal
Arquitetura + setup	5h	R\$ 300	R\$ 1.500
Domain + Application layer	8h	R\$ 300	R\$ 2.400
Infrastructure (OSRM, Nominatim, Traffic Engine)	10h	R\$ 300	R\$ 3.000
API routes + validação Zod + OpenAPI	5h	R\$ 300	R\$ 1.500

Item	Horas	Rate Sênior+IA (R\$/h)	Subtotal
Frontend: mapa MapLibre + tema dark	8h	R\$ 300	R\$ 2.400
Frontend: bottom sheet + stats + UX flow	8h	R\$ 300	R\$ 2.400
Docker compose + Dockerfiles + NestJS ref	4h	R\$ 300	R\$ 1.200
QA, debugging, ajustes de UX	6h	R\$ 300	R\$ 1.800
Documentação + README	2h	R\$ 300	R\$ 600
TOTAL MVP ATUAL	Aprox. 56h	—	R\$ 16.800

7.2 Comparação com Cenários de Evolução

O valor abaixo representa o custo de reprodução do produto em diferentes estágios de maturidade, sempre considerando desenvolvimento sênior + IA:

Estágio	Horas	Valor (Sênior+IA)	Prazo
MVP atual (este documento)	Aprox. 56h	R\$ 15.000 a 18.000	2 a 3 semanas
MVP + backend NestJS + Postgres	Aprox. 85h	R\$ 25.000 a 30.000	4 a 5 semanas
MVP + trânsito real + deploy produção	Aprox. 100h	R\$ 30.000 a 35.000	5 a 6 semanas
Produto vendável (+ auth + favoritos + PWA)	Aprox. 150h	R\$ 45.000 a 55.000	8 a 10 semanas
Produto com IA preditiva básica	Aprox. 230h	R\$ 70.000 a 85.000	12 a 16 semanas
Concorrente de Waze (escala completa)	Mais de 1000h	R\$ 300.000+	Mais de 6 meses

7.3 Avaliação para Cenário de R\$ 22.000

Considerando o orçamento de R\$ 22.000 mencionado, a análise de viabilidade é:

Cenário	Viável?	Justificativa
Reproduzir o MVP atual	SIM	Custo de reprodução aprox. R\$ 16.800, dentro do orçamento
MVP + backend NestJS rodando	SIM (justo)	R\$ 25 a 30k, requer negociação de escopo
MVP + trânsito real + deploy	MARGINAL	R\$ 30 a 35k, exige reduzir escopo em outro lugar
Produto vendável completo	NÃO	R\$ 45 a 55k, orçamento insuficiente
Produto com IA preditiva	NÃO	R\$ 70 a 85k, fora do orçamento

Recomendação Comercial

Para R\$ 22.000, o escopo honesto e entregável é: MVP atual (já desenvolvido) + backend NestJS rodando com persistência + deploy em VPS + documentação de handover. NÃO incluir IA real (manter slot de IA como diferencial arquitetural), NÃO incluir trânsito real (manter simulação), NÃO incluir app nativo. Entrega em 4 a 5 semanas. Payment milestone sugerido: 40% upfront, 30% na entrega do backend, 30% no deploy + handover.

8. Roadmap e Próximos Passos

8.1 Roadmap de Curto Prazo (4 a 6 semanas)

- Substituir API routes do Next.js pelo backend NestJS rodando em Docker
- Configurar PostgreSQL + PostGIS para persistência de rotas calculadas e histórico
- Integrar Redis para cache de respostas de rota (TTL 5 min por par origem-destino)
- Self-host OSRM com extract de São Paulo (sudeste-latest.osm.pbf)
- Deploy em VPS (DigitalOcean ou Hetzner) com docker compose up
- Documentação de handover + runbook de operação

8.2 Roadmap de Médio Prazo (8 a 12 semanas)

- Sistema de autenticação (NextAuth.js) com login social (Google ou Apple)
- Favoritos de destinos e histórico de rotas por usuário

- PWA com service worker para uso offline básico
- Integração de trânsito real via API (TomTom ou Here, custo a validar)
- Detecção de fechamento de vias e acidentes (slot já preparado no scorer)
- Painel administrativo para gestão de vias monitoradas

8.3 Roadmap de Longo Prazo (3 a 6 meses)

- Modelo de IA preditiva de trânsito (regressão sobre dados históricos coletados)
- Crowdsourcing real de trânsito (usuários reportam via app)
- Apps nativos iOS e Android via Capacitor ou React Native
- Otimização para veículos elétricos (slot já preparado no scorer)
- Integração com smart cities (semáforos inteligentes, dados abertos)
- Score de segurança viária (slot já preparado no scorer)

9. Conclusão e Recomendação Final

O TrafficMind em seu estado atual de MVP é um produto técnico viável e funcional. Ele demonstra end-to-end o fluxo de navegação inteligente com cálculo de rotas multi-critério, ranking por score transparente e interface mobile-first em português. A arquitetura Clean Architecture com slot de IA preparado é o maior diferencial técnico — permite evolução incremental sem reescrita.

Para o cenário de R\$ 22.000, a recomendação é fechar escopo honesto: entregar o MVP atual mais backend NestJS rodando e deploy em VPS. Isso custa aproximadamente R\$ 16.800 para reproduzir (já desenvolvido), deixando margem para as 25 a 30 horas adicionais de backend + deploy dentro do orçamento. Não prometer IA real, trânsito real ou app nativo — esses são incrementos que extrapolam o orçamento e exigiriam recontração.

O maior risco comercial é o cliente ter expectativas de "um Waze mais inteligente" sem escopo definido. Recomenda-se alinhar por escrito o que está incluído e o que não está, com demo do MVP atual como prova de conceito. O indicador visual de progresso (Passo 1, 2 e confirmação), o painel de score com breakdown transparente e a UI em português são os pontos de venda mais fortes para um cliente não-técnico.

Para o time técnico, os pontos de atenção são: o OSRM público não sustenta produção (precisa self-host), o trânsito simulado é apenas demonstração (precisa de feed real para produto vendável), e a UI atual é web-only (precisa de wrapper Capacitor para lojas de apps). Esses três itens são os próximos passos naturais após o fechamento do escopo de R\$ 22.000.